

EUROPYTHON 2026 · A PYTEST CASE STUDY

The hardest test suite I ever built

Grzegorz Kocjan · BeLazy.dev

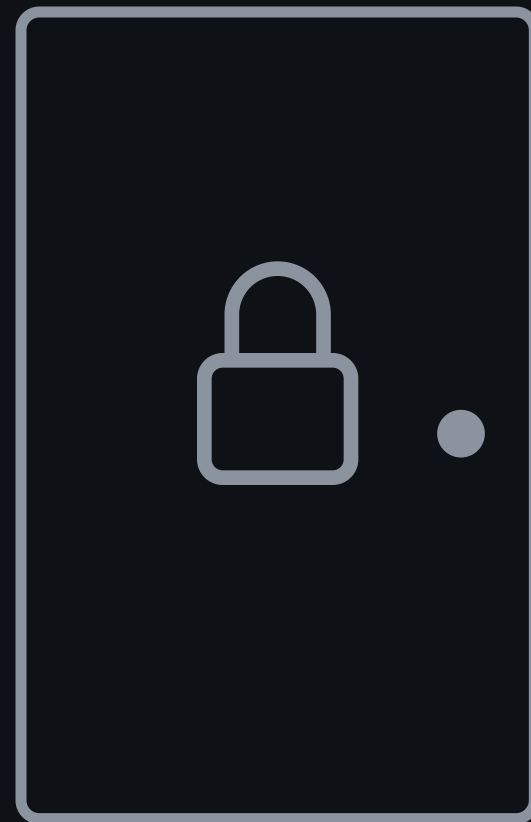
```
def test_car_has_correct_color(car_case, first_sighting):  
    assert first_sighting["color"] == car_case.current.color
```

2021 · COMPUTER VISION · 17 MONTHS IN PRODUCTION

Integration tests: 0

Unit tests: 96

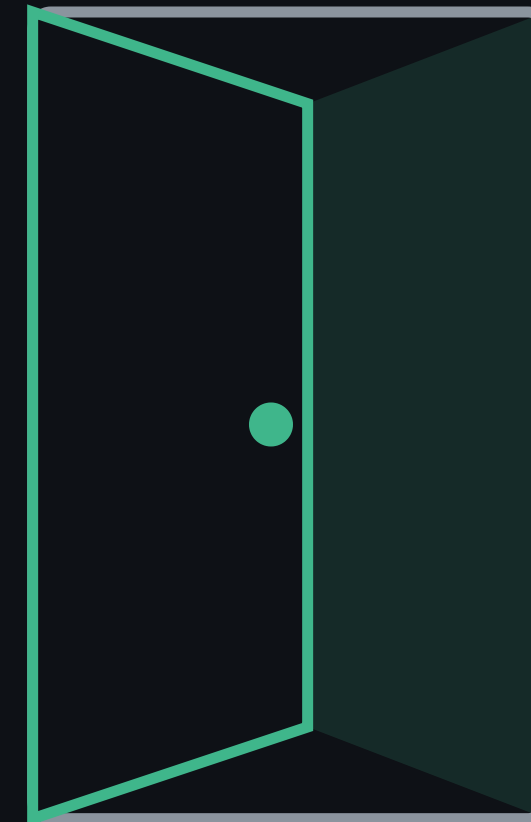
Tests that could catch a bug: 0



unit tests



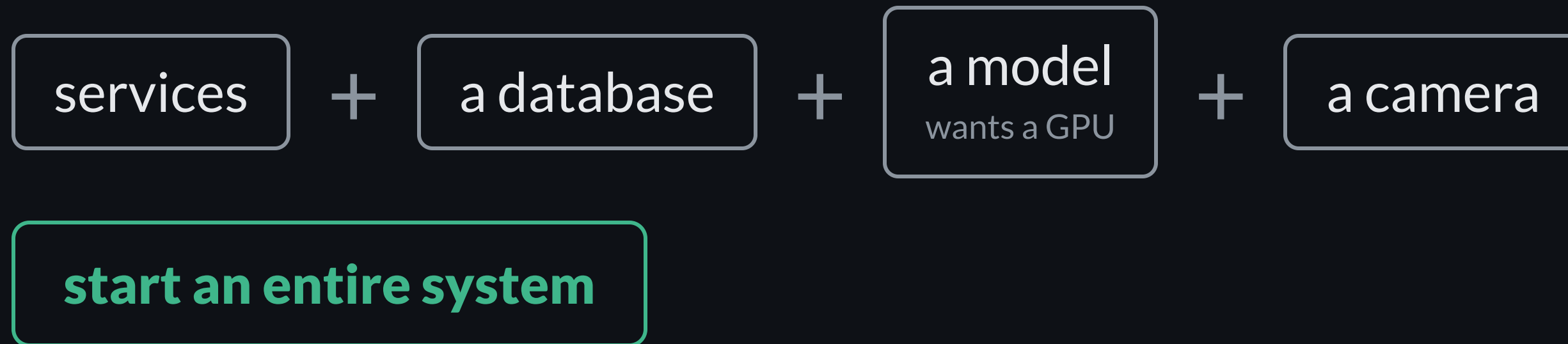
acceptance tests



integration tests

The only door that wasn't locked.

The *Arrange*



ARRANGE: PYTEST BRINGS THE SYSTEM UP

```
1 @pytest.fixture(scope="session")
2 def web_service(docker_services):
3     docker_services.start("web")           # docker compose up -d
4     port = docker_services.port_for("web", 80)
5     docker_services.wait_until_responsive(   # don't sleep – wait for ready
6         check=lambda: check_url(docker_services.docker_ip, port),
7         timeout=30.0, pause=0.5,
8     )
9     return f"http://{docker_services.docker_ip}:{port}"
```

THE REAL TRICK: THE CAMERA

recorded
frames

The system does

[WED • 4 PM]



```
timeout-minutes: 10 # a reasonable test suite
```

```
timeout-minutes: 2880 # this one
```


Your system doesn't have to be small.

It has to be startable.

the system starts itself ✓

the recording plays ✓

the tests run ✓

but...

MEANWHILE

24 hours later...

ONE LEVEL DOWN – INTO PYTEST'S BASEMENT

```
1 def pytest_generate_tests(metafunc):
2     recording_cases, vehicle_cases = load_cases(...) # our cases live in JSON
3
4     if "vehicle_case" in metafunc.fixturenames: # scope: ONE car
5         metafunc.parametrize("vehicle_case", vehicle_cases,
6                               ids=id_from_vehicle, indirect=True)
7
8     if "recording_case" in metafunc.fixturenames: # scope: WHOLE recording
9         metafunc.parametrize("recording_case", recording_cases,
10                              ids=id_from_recording, indirect=True)
```

the test picks its scope by signature alone – no decorators:

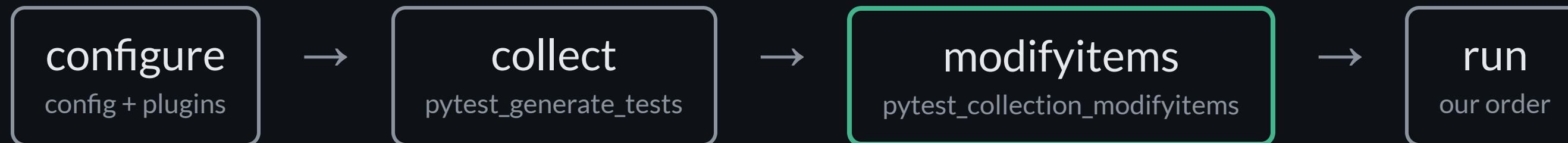
```
def test_vehicle_has_correct_plate(vehicle_case, first_sighting): ...
def test_no_misses_extras_or_late(recording_case, recording_stopped): ...
```


HERE'S WHAT PYTEST DOES BY DEFAULT

```
1 test_cars.py::test_color[car1]
2 test_cars.py::test_color[car2]
3 test_cars.py::test_color[car3]
4 test_cars.py::test_color[car4]
5 test_cars.py::test_color[car5]
6 test_cars.py::test_plate[car1]
7 test_cars.py::test_plate[car2]
8 test_cars.py::test_plate[car3]
9 test_cars.py::test_plate[car4]
10 test_cars.py::test_plate[car5]
```

10 tests collected – ordered by function, not by car

pytest's startup sequence



GROUP BY RECORDING · LIVE FIRST, END LAST

```
1 @pytest.fixture
2 def recording_stopped():
3     pass # empty fixture = a phase marker: "I run after the movie ends"
4
5 def pytest_collection_modifyitems(session, config, items):
6     grouped = defaultdict(lambda: {"live": [], "end": []})
7     for item in items:
8         case = item.callspec.params.get("vehicle_case") \
9             or item.callspec.params.get("recording_case")
10        phase = "end" if "recording_stopped" in item.fixturenames else "live"
11        grouped[case.recording][phase].append(item)
12
13    items[:] = chain(*[*g["live"], *g["end"]] for g in grouped.values())
```


**Every pytest tutorial says:
a test must assert.**

Half of this suite disagrees.

AFTER MODIFY ITEMS: THE TESTS COME ALIVE

```
1 test_cars.py::test_start_recording[rec1]
2 test_cars.py::test_color[car1]
3 test_cars.py::test_plate[car1]
4 test_cars.py::test_color[car2]
5 test_cars.py::test_plate[car2]
6 test_cars.py::test_color[car3]
7 test_cars.py::test_plate[car3]
8 test_cars.py::test_color[car4]
9 test_cars.py::test_plate[car4]
10 test_cars.py::test_color[car5]
11 test_cars.py::test_plate[car5]
12 test_cars.py::test_wait_for_recording_stop[rec1]
```

THE WORLD I CAME FROM

```
assert user_count == 1
```

the world where *correct* means *equal*

I lied.

```
def test_car_has_correct_color(car_case, first_sighting):  
    if first_sighting["color"] != car_case.current.color:  
        warnings.warn("Wrong color")    # it cannot fail the build
```

4 Levels of Trust

1 · HARD ASSERT
direction · timing · plate (≤ 1 char off)

2 · WARNINGS.WARN
color · make · model

3 · ?

4 · ?

PLATE	BHA-215
COLOR	navy
MAKE	Honda
TIME IN	16:05

PLATE = the bill.
COLOR · MAKE = a hint.

nobody ever corrected those

The business, copied into assertions.

The suite learned to trust less — one assert at a time

GIT • JUL 30, 2021

```
- assert first_sighting["color"] == case.color  
+ warnings.warn("Wrong color")
```

GIT • OCT 4, 2021

```
- assert first_sighting["make"] == case.make  
- assert first_sighting["model"] == case.model  
+ warnings.warn("Wrong make or model")
```

2 tests: FAILED

both cars detected perfectly ✓

a wide entrance — two cars, side by side

expected: [A, B] vs reported: [B, A] — the order flips



Three suitcases are circling.
You take yours — check it, move on.
Not yours? Someone else will claim it.
Each test waits for *its* car.

CARS MATCHMAKING

```
1 def _weight(expected, received):
2     if expected.direction != received.direction:
3         return MAX_WEIGHT          # irreconcilable
4
5     distance = edit_distance(expected.plate, received.last_plate)
6     if distance > MAX_EDIT_DISTANCE:
7         return MAX_WEIGHT
8
9     lateness = max(0, abs(received.frame_time - expected.appear_at) - 2_000)
10    return distance + lateness / 10_000    # 10 seconds late = 1 typo
11
12 cost_matrix = [[_weight(e, r) for r in received] for e in expected]
13 rows, cols = linear_sum_assignment(cost_matrix)    # Hungarian algorithm (scipy)
```

10 seconds late = 1 typo. MAX_WEIGHT = simply forbidden.

expected, no event → **misses**

event, no car → **extras** (the ghosts)

matched, too slow → **late**

1 • hard assert

2 • warn

3 • per recording

4 • ?

1 · hard assert

2 · warn

3 · per recording

4 · ?

LEVEL 3 — STATISTICS PER RECORDING

```
1 from pytest_check import check          # soft asserts — test runs to the end
2
3 def test_no_misses_extras_or_late(recording_case, event_store,
4                                   recording_stopped, results_bag):
5     stats = calculate_misses_extras_and_late(expected, received)
6     results_bag.statistics = stats        # collect — the report comes later
7
8     with check:
9         assert stats.misses == [], f"{len(stats.misses)} missed"
10    with check:
11        assert stats.extras == [], f"{len(stats.extras)} ghosts"
12    with check:
13        assert stats.late == [], f"{len(stats.late)} late"
```

1 · hard assert

2 · warn

3 · per recording

4 · ?

REMEMBER THE WARNINGS?

```
def test_car_has_correct_color(car_case, first_sighting):  
    if first_sighting["color"] != car_case.current.color:  
        warnings.warn("Wrong color")    # it cannot fail the build
```

But a warning cannot catch the trend.

1 • hard assert

2 • warn

3 • per recording

4 • aggregate

1 · hard assert

2 · warn

3 · per recording

4 · aggregate

LEVEL 4 — MY FAVORITE TEST: IT TESTS NO CAR AT ALL

```
1 from pytest_harvest import get_fixture_store # store from every test
2
3 def test_accuracy(request, reports_path):
4     store = get_fixture_store(request) # all the results_bags
5     stats_by_recording = {
6         bag["recording"]: bag["statistics"]
7         for bag in store["results_bag"].values() if "statistics" in bag
8     }
9     print(summary_report(stats_by_recording)) # table per recording + TOTAL
10
11     total = sum(stats_by_recording.values(), Statistics())
12     assert total.accuracy >= 0.85, f"{total.accuracy:.0%} below the 85% bar"
```

1 · hard assert

2 · warn

3 · per recording

4 · aggregate

LEVEL 4 — MY FAVORITE TEST: IT TESTS NO CAR AT ALL

```
1 from pytest_harvest import get_fixture_store # store from every test
2
3 def test_accuracy(request, reports_path):
4     store = get_fixture_store(request) # all the results_bags
5     stats_by_recording = {
6         bag["recording"]: bag["statistics"]
7         for bag in store["results_bag"].values() if "statistics" in bag
8     }
9     print(summary_report(stats_by_recording)) # table per recording + TOTAL
10
11     total = sum(stats_by_recording.values(), Statistics())
12     assert total.accuracy >= 0.90, f"{total.accuracy:.0%} below the 90% bar"
```

One wrong color: a warning. A trend: a failed build.

What we actually built

test file: 401 lines · 21 tests

conftest: 744 lines — 2× more infrastructure, on purpose

× double parametrization · 203 recordings · 2 713 events → ~39 000 test items

...from **four pages** of code.

bug reported → pull the recording → JSON case

→ fix → smoke tests

→ long run overnight

→ next day: ship

~~deploy & wait for the phone to ring~~

How long did it take to **build** this? (not to run it — to build it)

4 months

We built the smallest thing that gave feedback — and
we continue improving it.

ARRANGE • ACT • ASSERT — ALL IN THE BODY

```
1 def test_recognize_car():  
2     frame = load_frame("bha-215.png")           # Arrange  
3     sighting = recognize(frame)                   # Act  
4     assert sighting.plate == "BHA-215"           # Assert
```

Simple? Then this is perfect.

THE MOVE WE ALL KNOW: ARRANGE → FIXTURE

```
1 @pytest.fixture
2 def frame():
3     return load_frame("bha-215.png")          # Arrange
4
5 def test_recognize_car(frame):
6     sighting = recognize(frame)                # Act
7     assert sighting.plate == "BHA-215"        # Assert
```

THE ONE FEWER REACH FOR: ACT → FIXTURE

```
1 @pytest.fixture(scope="module")
2 def frame():                                # Arrange
3     return load_frame("bha-215.png")
4
5 @pytest.fixture(scope="module")
6 def sighting(frame, recognize_spy):          # Act — runs once per module
7     recognize_spy = Mock(wraps=recognize)
8     return recognize_spy(frame)
9
10 def test_plate(sighting):
11     assert sighting.plate == "BHA-215"
12
13 def test_color(sighting):
14     assert sighting.color == "navy"
15
16 def test_act_ran_once(recognize_spy):
17     assert recognize_spy.call_count == 1      # proof: 2 tests, one Act
```

Hide the complexity in the fixtures. Keep the tests down to a few lines.

**"I made this longer only
because...
...I had no time to make it
shorter."**

— Blaise Pascal, 1657

The simplest tests I ever wrote.
The hardest suite I ever built.

Go build yours.



belazy.dev/talks/the-hardest-test-suite-i-ever-built
Grzegorz Kocjan